FACULTAD DE INFORMÁTICA

UNIVERSIDAD NACIONAL DE LA PLATA

I118 Taller de Proyecto II

Entrega 2: Informe de avance de octubre

# LoRa OTA Update

Grupo J5
Bermúdez, Lautaro Joaquín[1] (01399/0)
Majoros, Lorenzo[2] (03296/1)
Seery, Juan Martín[2*] (03471/9)

11/10/2026

[1]lautarojbermudez@gmail.com
[2]{lorenzomajoros,juan.seery}@alu.ing.unlp.edu.ar
[*]Autor responsable del informe

---

## Índice

Nomenclatura ..... 2
1. Descripción general del proyecto ..... 3
1.1. Introducción ..... 3
1.2. Objetivo ..... 3
1.2.1. Requerimientos ..... 3
1.2.2. Métricas ..... 5
1.3. Cambios respecto al Plan de Proyecto ..... 5
2. Materiales y presupuesto ..... 6
3. Esquema gráfico del proyecto ..... 7
3.1. Esquema del sistema ..... 7
3.2. Casos de uso actuales ..... 7
3.3. Conexiones físicas de las placas ..... 8
4. Grado de avance, problemas y soluciones ..... 9
4.1. Estado de cada parte ..... 9
4.2. Informe de avance de octubre ..... 11
4.2.1. Mediciones realizadas ..... 12
4.2.2. Prueba básica en la mesa ..... 12
4.2.3. Prueba de campo ..... 12
4.2.4. Prueba de capacidad ..... 14
4.3. Documentación del código ..... 16
4.3.1. Código que maneja el hardware ..... 16
4.3.2. Trama y protocolo de enlace ..... 17
4.3.3. Código que maneja las comunicaciones con la computadora ..... 17
4.3.4. Interfaz web y software en la computadora ..... 18
4.4. Problemas encontrados y soluciones ..... 19
5. Documentación relacionada ..... 20
5.1. Fotos de las partes construidas ..... 20
5.2. Capturas del panel de control ..... 20
5.3. Enlaces ..... 23
Bibliografía ..... 24
A. Datos de las mediciones ..... 25
A.1. Prueba de campo de octubre ..... 25
A.2. Prueba de capacidad de octubre ..... 25

## Nomenclatura

| | |
|---|---|
| OTA | (*Over-the-air*) Actualización de software distribuida por un enlace inalámbrico |
| MCU | (*Micro-controller unit*) Microcontrolador |
| VM | (*Virtual machine*) Intérprete de instrucciones |
| LoRa | (*Long Range*) Modulación de radio de largo alcance |
| SF | (*Spreading factor*) Factor de dispersión |
| BW | (*Bandwidth*) Ancho de banda del canal LoRa |
| CR | (*Coding rate*) Tasa de código de la corrección de errores |
| PDR | (*Packet delivery ratio*) Fracción de paquetes entregados |
| RTT | (*Round-trip time*) Tiempo de ida y vuelta |
| RSSI | Indicador de intensidad de señal recibida |
| SNR | Relación señal a ruido |
| HMAC | Código de autenticación de mensajes basado en una función de hash |
| LNA | (*Low-noise amplifier*) Amplificador de bajo ruido |
| TCXO | Oscilador a cristal compensado en temperatura |
| PWA | (*Progressive web app*) Aplicación web instalable que funciona sin conexión |
| ADR | (*Architecture decision record*) Registro de una decisión de arquitectura |
| NVS | (*Non-volatile storage*) Memoria no volátil del ESP32 |

# 1. Descripción general del proyecto

## 1.1. Introducción

Las redes de dispositivos embebidos distribuidos a lo largo de un territorio, como sensores agrícolas, estaciones meteorológicas o nodos de telemetría, comparten un problema práctico: una vez instalados, actualizar el software que ejecutan exige que un operario se movilice hasta cada uno de los nodos. La actualización remota OTA resulta una solución atractiva a ese problema. Típicamente, las actualizaciones OTA se realizan sobre WiFi o red celular, como es el caso de los teléfonos celulares, pero eso limita el alcance y la complejidad de los dispositivos. LoRa, en cambio, ofrece kilómetros de alcance a cambio de tasas de datos de unos pocos kilobits por segundo y paquetes de a lo sumo 255 bytes.

El problema tiene antecedentes en las redes de sensores inalámbricos, donde protocolos como Deluge [1] y Trickle [2] resolvieron la propagación de código sobre radiofrecuencia de baja potencia mediante diseminación epidémica y reenvío selectivo. Este proyecto toma esas ideas y las lleva a un escenario acotado — una red de microcontroladores ESP32 con antenas LoRa donde se distribuye un *símil firmware*: programas para una VM ligera que cada nodo ejecuta.

En su propuesta más básica, este proyecto busca abarcar dos MCU con módulos LoRa y una micro VM para procesamiento numérico básico en cada uno. Cualquier nodo puede conectarse a una computadora y recibir la nueva versión del *firmware* y retransmitirla a su par, además de coordinar entre ambos para ejecutar la versión nueva. El grupo propone además extender el sistema a más de dos nodos, de modo que la retransmisión punto a punto sea el caso particular de un protocolo de inundación en malla. Esa extensión está planteada como objetivo secundario y condicionada a la disponibilidad de hardware adicional por parte de la cátedra.

## 1.2. Objetivo

El objetivo general del presente proyecto es diseñar, implementar y evaluar un sistema de actualización OTA de programas ejecutables sobre una red descentralizada de nodos ESP32 con LoRa, en el que cada nodo corre los programas sobre una micro VM propia.

En particular, los **objetivos principales** se desglosan en:

a) especificar e implementar una micro VM de bytecode para procesamiento numérico entero, con su formato de programa, su herramienta de ensamblado y un intérprete de referencia que permita verificar la correctitud de la ejecución en los nodos;
b) especificar e implementar un protocolo de distribución OTA sobre LoRa entre dos nodos, con fragmentación, detección y reparación de pérdidas y activación coordinada de la versión nueva con confirmación al origen;
c) permitir que cualquier nodo actúe como origen de una actualización al conectarlo por USB o por WiFi[1] a la computadora, sin roles fijos;
d) implementar las herramientas necesarias para analizar métricas de desempeño y correctitud y analizar los resultados;
e) desarrollar un panel de control que se conecte por USB o por WiFi[1] a cualquier nodo para cargar programas, observar el estado de la red y visualizar las métricas;
f) asegurar el origen de los programas mediante firma digital de la imagen, autenticación de cada paquete para evitar manipulaciones externas.

Además, se define como **objetivos secundarios**:

a) extender el protocolo a tres o más nodos en malla, con retransmisión multisalto, control de colisiones entre retransmisores y descubrimiento automático de los nodos de la red mediante mensajes periódicos de presencia;
b) extender las métricas definidas a una red en malla y analizar los resultados;
c) armar un nodo portable alimentado a batería que permita conectarse a la red desde cualquier punto.

### 1.2.1. Requerimientos

**Requerimientos funcionales**

[1] Originalmente solo se dipuso por USB, se agregó la funcionalidad de WiFi para facilitar las pruebas de campo.

RF1) **Ejecución continua**. Cada nodo debe ejecutar de forma permanente el programa que tiene activo, y sus resultados deben poder observarse desde una PC conectada a cualquier nodo de la red.

RF2) **Carga**. Un operador debe poder cargar una nueva versión de programa desde una PC conectada por USB o por WiFi[1] a cualquier nodo, sin que ese nodo requiera un rol, un firmware o una configuración distintos de los demás.

RF3) **Distribución**. Toda versión cargada en un nodo debe llegar completa e idéntica a todos los demás nodos de la red, aun cuando el enlace de radio pierda paquetes.

RF4) **Activación coordinada**. Ningún nodo debe ejecutar la versión nueva antes de que el nodo origen lo ordene y el origen solo debe ordenarlo cuando todos los nodos conocidos dispongan de la versión completa. Hasta ese momento, cada nodo debe seguir ejecutando su versión anterior sin interrupciones.

RF5) **Confirmación**. El nodo origen debe informar al operador cuándo todos los nodos de la red quedaron ejecutando la versión nueva y, vencido un plazo, cuáles no lo hicieron. El operador debe poder forzar la activación en los nodos que ya disponen de la versión o abortar la actualización.

RF6) **Convergencia**. Si se inician dos actualizaciones de forma concurrente desde nodos distintos, la red debe converger a una única versión, la más nueva, y nunca deben coexistir dos programas distintos bajo el mismo número de versión. Un conflicto de ese tipo debe ser visible para el operador.

RF7) **Aislamiento de fallas**. Un programa defectuoso (lazo infinito, división por cero, acceso inválido) no debe impedir que el nodo siga participando de la red ni recibiendo actualizaciones. El error debe quedar registrado y ser visible para el operador.

RF8) **Verificabilidad**. Debe poder comprobarse desde la PC que el resultado que produce cada nodo es el esperado para el programa y la versión que tiene activos.

RF9) **Reinicio**. Un nodo que se reinicia debe quedar en un estado conocido, informarlo al resto de la red y volver a la versión vigente de forma automática, sin intervención del operador.

RF10) **Observabilidad**. Desde una PC conectada a cualquier nodo, el operador debe poder conocer la versión que ejecuta cada nodo, sus últimos resultados, sus errores, el progreso de la actualización en curso y la calidad de los enlaces de radio.

RF11) **Registro de métricas**. El sistema debe registrar por sí mismo los datos necesarios para calcular las métricas de la Tabla 1 y permitir exportarlos.

RF12) **Autenticidad**. Un nodo solo debe instalar versiones cuya procedencia de un operador autorizado y cuya integridad pueda verificar. Debe descartar sin efecto alguno cualquier imagen o mensaje que no supere esa verificación. La capacidad de autorizar una versión debe residir en el operador y no en los nodos: un nodo comprometido no debe poder hacer que los demás acepten un programa.

RF13) **Alcance multisalto** *(secundario)*. Una actualización debe llegar a nodos que están fuera del alcance de radio directo del origen, siempre que exista una cadena de nodos intermedios que los conecte.

RF14) **Descubrimiento** *(secundario)*. La red debe determinar por sí misma qué nodos la integran, sin una lista configurada de antemano, y reflejar la aparición o la desaparición de un nodo dentro de un plazo acotado.

**Requerimientos no funcionales y de tiempo real**

RNF1) **Tiempo**. No hay requerimientos de tiempo real estricto. Sí se exige que la ejecución de un programa nunca demore la atención de la radio más allá de una cota fija, de modo que ningún paquete se pierda por culpa del programa, y que una actualización completa entre dos nodos se complete en un tiempo compatible con una demostración en vivo.

RNF2) **Autonomía**. El sistema debe funcionar sin conexión a Internet. Una actualización ya iniciada debe completarse aunque la PC se desconecte del nodo origen.

RNF3) **Uniformidad**. Todos los nodos deben correr el mismo código fuente, compilado una vez por cada modelo de placa[2], sin variantes por rol, y el diseño debe admitir al menos cuatro nodos sin cambios en él.

[2]Originalmente era exactamente el mismo modelo, pero se agregaron placas con ligeras diferencias.

RNF4) **Entregables**. El firmware se entrega como proyecto de Arduino, según lo requiere la cátedra. El panel de control debe poder ejecutarse sin conexión a Internet en una PC con navegador basado en Chromium y conectarse a un nodo por USB o por WiFi[1].

RNF5) **Regulación**. El sistema debe operar en la banda de uso libre de 915 a 928 MHz vigente en Argentina según la Res. ENACOM 4653/2019 [3], lo que con estas placas implica canales de 500 kHz[3].

### 1.2.2. Métricas

Aparte del funcionamiento intrínseco de lo construido, se realizará especial énfasis en medir el rendimiento logrado. Para ello, se definen distintas métricas que la red podrá medir por sí misma a partir del flujo de eventos que emiten los nodos. Las mismas se listan en la Tabla 1; cómo se mide cada una con lo construido está en la Sección 4.2.1.

**Tabla 1.** Métricas definidas para el sistema.

| Métrica | Definición | Unidad | Método |
|---|---|---|---|
| M1. Tiempo de convergencia | Desde que la PC entrega la imagen al origen hasta que el último nodo confirma la activación | s | Marcas de tiempo del origen; en función del tamaño de imagen y de la cantidad de nodos |
| M2. Eficiencia de la inundación | Transmisiones totales en la red dividido el mínimo teórico (fragmentos por cantidad de nodos menos uno) | — | Contadores de transmisión de cada nodo reportados en los mensajes de presencia |
| M3. Pérdida y reparación | Porcentaje de fragmentos faltantes en el primer reporte de cada nodo y cantidad de rondas de reparación | % | Bitmaps de los reportes recibidos por el origen |
| M4. Latencia de ida y vuelta | Tiempo entre un mensaje dirigido a un nodo y su respuesta | ms | Marcas de tiempo del origen; por cantidad de saltos |
| M5. Correctitud de la VM | Ticks cuya salida coincide con el intérprete de referencia sobre el total | % | Comparación en el panel entre lo reportado por los nodos y la ejecución de referencia |
| M6. (*secundaria*) Calidad de enlace | RSSI y SNR de los mensajes de presencia de cada vecino | dB | Lectura del transceptor en cada recepción |
| M7. (*secundaria*) Recursos | Uso de RAM y flash del firmware | bytes | Reporte del compilador y del sistema en ejecución |

## 1.3. Cambios respecto al Plan de Proyecto

Hubo dos grandes cambios respecto al Plan de Proyecto presentado en agosto. El primero respecta a una nueva funcionalidad recomendada por el docente: que el panel de control pueda conectarse a un nodo por WiFi además de por USB para facilitar las pruebas de campo. El segundo es que se agregaron placas Heltec WiFi LoRa 32 V4.3 al alcance del proyecto, con un transceptor SX1262 y una etapa de RF externa, además de las placas V2 con SX1276 previstas originalmente.

Otras decisiones de diseño se tomaron en conjunto con el docente y se registraron como ADR en el repositorio, siguiendo la metodología de ***spec-driven development*** en conjunto a agentes de inteligencia artificial. No se han tomado grandes decisiones que difieran de lo previsto de la propuesta de agosto, pero algunas sutilezas se especifican en la carpeta `docs/adrs` del repositorio y en la bitácora.

[3] Originalmente no se precisó la resolución ni el ancho de canal, es un agregado del avance de octubre.

## 2. Materiales y presupuesto

La Tabla 2 lista los materiales acordados con la cátedra. Los precios de las placas y de las antenas son los de la tienda oficial de Heltec sin envío ni impuestos de importación. El equivalente en pesos usa la cotización aproximada de $ 1.500. El software es libre y sin costo y la publicación del panel usa el plan gratuito de Cloudflare Workers, que sirve activos estáticos sin límite de tráfico y sin suscripción.

**Tabla 2.** Materiales del proyecto. «Sí» en «Disponible» significa que el grupo ya lo tiene.

| **Elemento** | **Cantidad** | **Costo unitario** | | **Costo total** | | **Disponible** | **Referencia** |
|---|---|---|---|---|---|---|---|
| | | USD | ARS | USD | ARS | | |
| Heltec WiFi LoRa 32 V2 (ESP32 y SX1276) | 2 | 17,90 | 26.850,00 | 35,80 | 53.700,00 | Sí | heltec.org, en discontinuación |
| Heltec WiFi LoRa 32 V4.3 (ESP32-S3 y SX1262) | 3 | 19,90 | 29.850,00 | 59,70 | 89.550,00 | Sí | heltec.org |
| Cable microUSB de datos | 2 | — | — | — | — | Sí | De uso general, ya disponible |
| Cable USB-C de datos | 3 | — | — | — | — | Sí | De uso general, ya disponible |
| Batería de litio 3,7 V = 1000 mAh | 5 | — | — | — | — | No | *secundario* |
| Publicación del panel: Cloudflare Workers, plan gratuito | 1 | 0,00 | 0,00 | 0,00 | 0,00 | Sí | developers.cloudflare.com |
| | | | | **95,50** | **143.250,00** | | |

La cátedra aportó con los kits completos de placas V2 mientras que el grupo consiguió por su cuenta las placas V4.3 para completar el objetivo de nodos conectados por mallas. Falta comprar las baterías que son necesarias para el objetivo secundario de un nodo portable.

# 3. Esquema gráfico del proyecto

## 3.1. Esquema del sistema

La Figura 1 esquematiza todos los dipositivos en juego. Lo que se abarca en el informe se dibuja en trazo continuo: el enlace de radio con la trama autenticada, los servicios de presencia, eco y corridas, la consola por USB y por WiFi y el texto de depuración en el OLED. La micro VM y el protocolo OTA van en trazo discontinuo, igual que los nodos adicionales de la malla.

La computadora del operador corre el panel de control en un navegador. Llega a un nodo por el puerto serie USB (Web Serial) o asociándose al punto de acceso WiFi del nodo, `J5-<id>`, con un WebSocket. Los nodos nunca se comunican entre sí por WiFi.

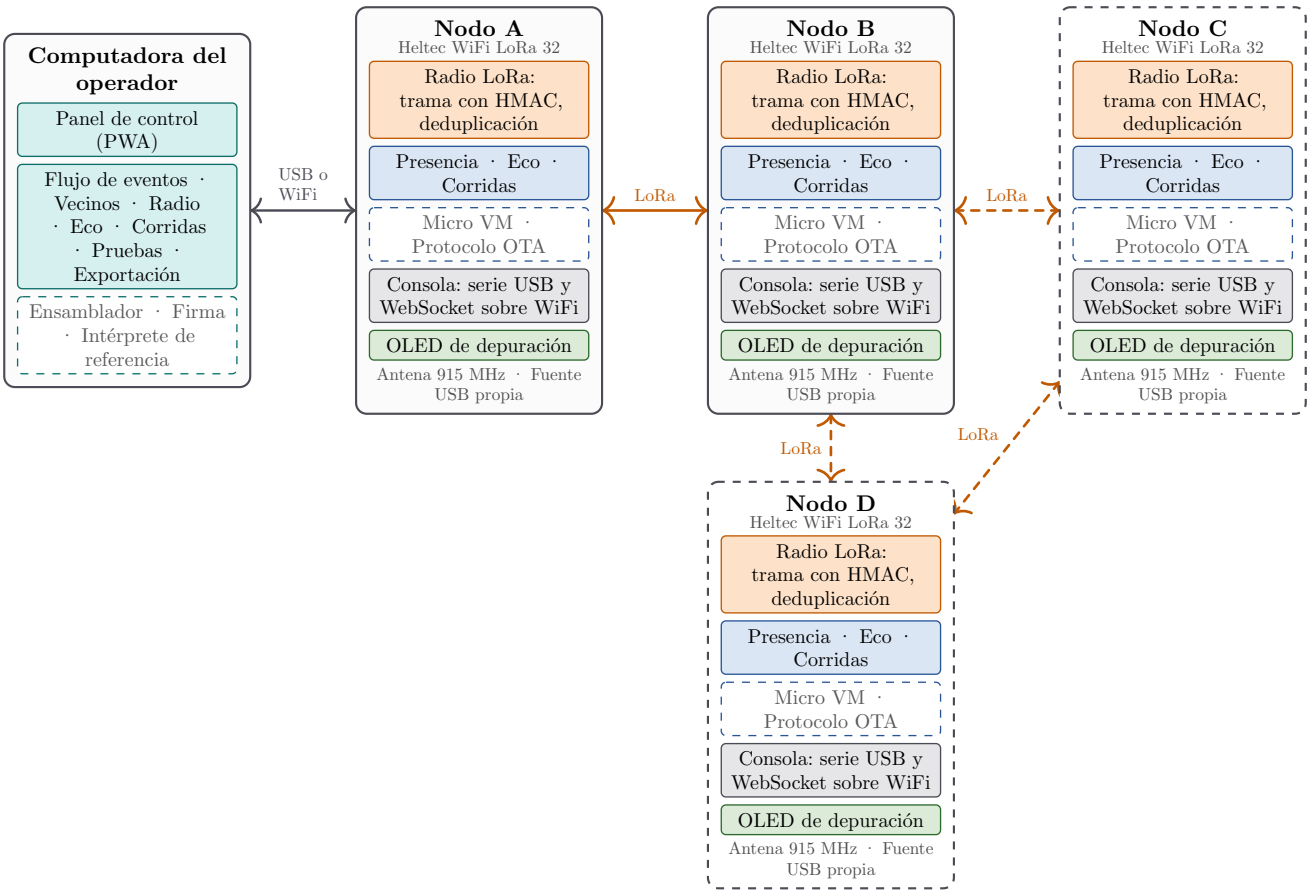



**Figura 1.** Esquema del sistema.
En trazo continuo, lo construido a octubre; en trazo discontinuo, lo previsto para noviembre y la extensión a malla.

## 3.2. Casos de uso actuales

Desde el panel, con uno o dos nodos conectados, el operador puede:

a) **Observar la red**: ver el flujo de eventos de cada nodo, su tabla de vecinos con RSSI y SNR en los dos sentidos y sus parámetros de radio.
b) **Pedir ecos**: un nodo pide un eco a otro y el panel muestra el RTT y la calidad de ida y de vuelta.
c) **Transmitir corridas**: un nodo transmite una cantidad de paquetes de prueba de un largo e intervalo dados; cada receptor informa la PDR y el RSSI medio, mínimo y máximo.
d) **Prueba de capacidad**: el panel recorre una matriz de modulaciones y largos sobre dos nodos y calcula el *goodput* y la PDR de cada punto contra la cota de Semtech.
e) **Prueba de comunicación**: en campo, con un operador en cada nodo, los dos nodos acuerdan una modulación y uno pide al otro una serie de ecos cortos para medir alcance, RSSI y SNR.
f) **Exportar**: cada medición queda en una carpeta con su ficha, todos los eventos y un resumen por corrida.

### 3.3. Conexiones físicas de las placas

Todo el hardware de un nodo está integrado en la placa Heltec: el microcontrolador, el transceptor LoRa, el OLED, el puente USB y el cargador de la batería. Las únicas conexiones externas son el cable USB a la computadora o a la fuente, la antena en el conector IPEX y, para el objetivo secundario, la batería en el conector SH1.25-2. La Figura 2 y la Figura 3 muestran cómo se conectan internamente las partes que maneja el firmware, con los números de GPIO que define la variante de cada placa en el core de Arduino para ESP32 [4].

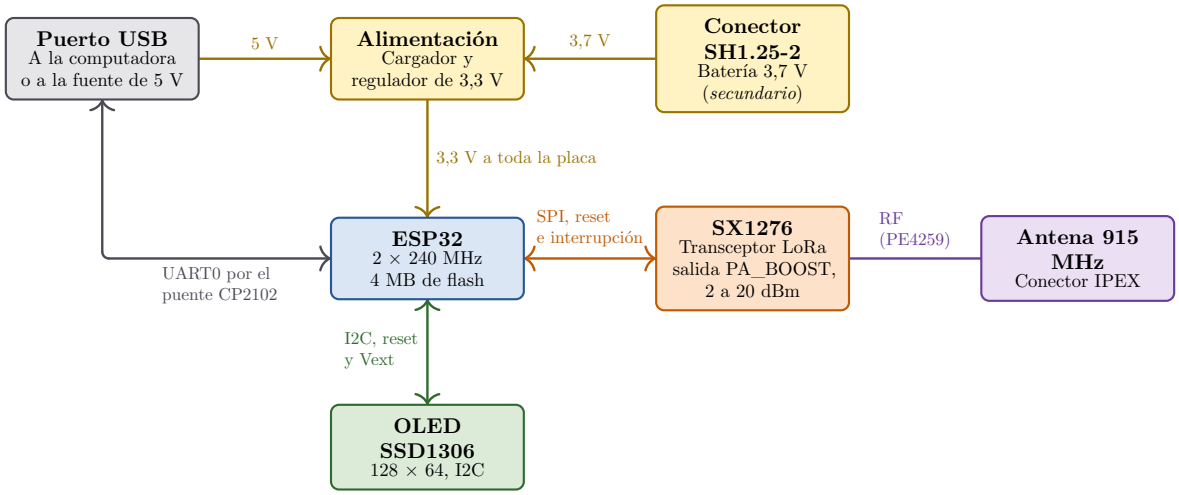



**Figura 2.** Conexiones internas de la Heltec WiFi LoRa 32 V2 que usa el firmware.

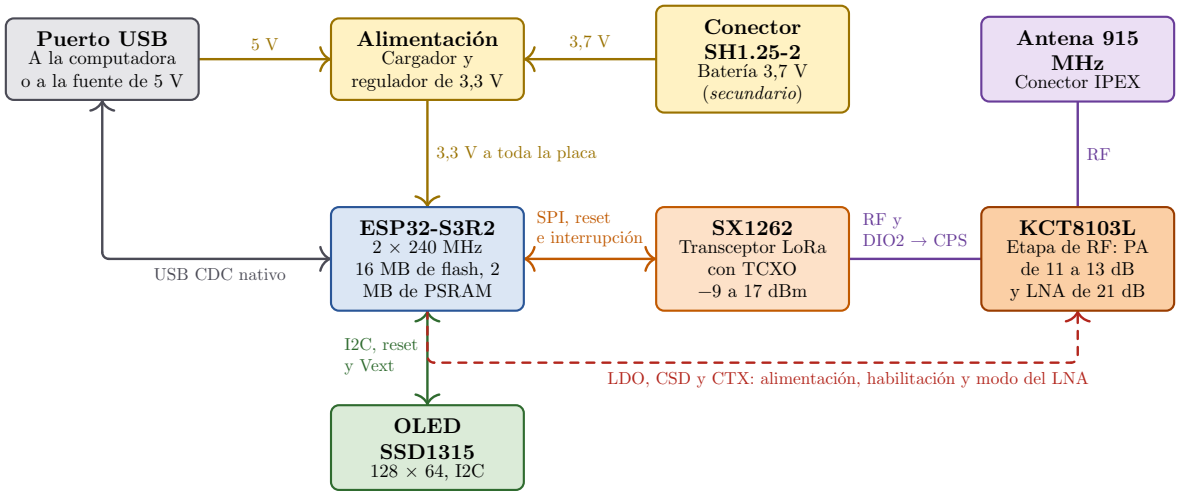



**Figura 3.** Conexiones internas de la Heltec WiFi LoRa 32 V4.3 que usa el firmware.

En la V4.3, la etapa de RF KCT8103L se controla con tres GPIO del ESP32-S3 y con la línea DIO2 del SX1262: el firmware la alimenta, la habilita y decide con CTX si en recepción interviene el LNA o queda puenteado, y el transceptor sube CPS solo mientras transmite. Con el LNA activo, una señal de más de unos −21 dBm en la antena satura el SX1262 y puede dañarlo; sobre la mesa, a un metro, una V2 a 20 dBm llega con unos −8 dBm. Por eso la V4.3 arranca con el LNA puenteado y el panel no permite cambiarlo.

# 4. Grado de avance, problemas y soluciones

## 4.1. Estado de cada parte

La Tabla 3 resume el avance de cada parte del proyecto respecto del alcance total del plan. Los porcentajes son aproximados. La Figura 4 ubica el estado en el cronograma del plan.

**Tabla 3.** Grado de avance por parte.

| Área | Parte | Avance | Comentario |
|---|---|---|---|
| Hardware | Placas y antenas | 100 % | Dos V2 y dos V4.3 en mano y probadas |
| | Alimentación independiente de cada nodo | 0 % | Sin baterías, los nodos se alimentan por USB; los consumos no se midieron |
| | Batería y nodo portable (*secundario*) | 0 % | *Previsto* para la entrega final |
| Firmware | Trama autenticada, identidad, deduplicación y vecinos | 100 % | |
| | Radio: tiempo de aire medido, soporte de V2 y V4.3 | 100 % | |
| | Presencia (`HELLO`), eco y corridas de prueba | 100 % | |
| | Consola por serie y por WiFi, eventos JSON, OLED | 100 % | |
| | Micro VM | 0 % | *Previsto* para el siguiente informe de avance |
| | Protocolo OTA: transferencia, reparación, activación | 0 % | *Previsto* para el siguiente informe de avance |
| | Retransmisión en malla (*secundario*) | 0 % | *Previsto* para el siguiente informe de avance |
| Panel | Conexión por USB y por WiFi, flujo de eventos, vecinos, radio | 100 % | |
| | Eco, corridas, prueba de capacidad, prueba de comunicación, exportación | 100 % | |
| | Publicación como PWA | 100 % | En https://j5-lora-ota.finisterre.app |
| | Ensamblador, firma, intérprete de referencia, carga de programas | 0 % | *Previsto* para el siguiente informe de avance |
| Mediciones | Tiempo de aire, RTT, RSSI y SNR sobre la mesa | 100 % | Sección 4.2.1 |
| | Prueba de comunicación en campo | 100 % | Sección 4.2.3 |
| | Barrido de potencia | 100 % | Sección 4.2.3 |
| | Prueba de capacidad a 500 kHz | 100 % | Sección 4.2.4 |
| | M1 a M3 y M5 | 0 % | Requieren la transferencia y la VM |

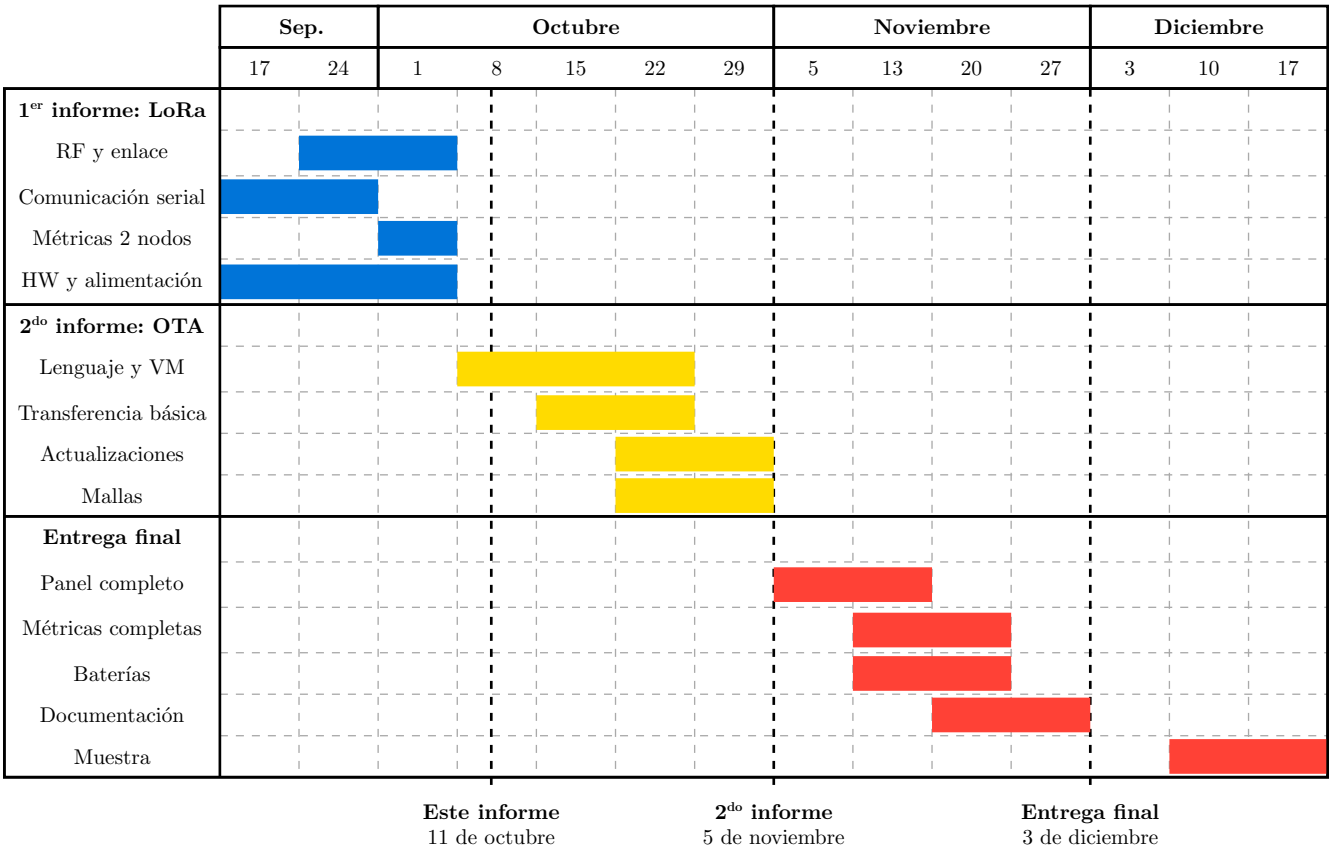



**Figura 4.** Cronograma del plan, con la fecha de este informe.

## 4.2. Informe de avance de octubre

El plan comprometía para este informe (E1, §5.1) los puntos de la Tabla 4.

**Tabla 4.** Cumplimiento de lo planificado para octubre.

| Área | Comprometido en el plan | Cumplido | Observaciones |
|---|---|---|---|
| Radio | Enlace LoRa operativo entre dos nodos con la cabecera definida, deduplicación y `HELLO` con versión y RSSI | Sí | Trama de 16 bytes de sobrecarga con HMAC-SHA256; filtro de repetidos por origen, época y secuencia; `HELLO` con la tabla de vecinos, verificable con el comando `resend` |
| | Tiempo de aire medido y comparado con el calculado | Sí | 15,669 ms medidos contra 15,424 ms calculados para un `HELLO` de 26 bytes (Sección 4.2.1) |
| | Parámetros de radio fijados | Sí | 915,9 MHz, SF7, 500 kHz, CR 4/5, preámbulo de 8, palabra 0x12, +4 dBm; se cambian en caliente con el comando `radio` |
| Consola | Flujo de eventos por serie legible desde el monitor del IDE de Arduino | Sí | Eventos JSON de una línea y texto de depuración, a 921600 baudios |
| | Panel mínimo: conexión por Web Serial y flujo de eventos | Sí, superado | Además: WiFi, vecinos, radio, eco, corridas, pruebas de capacidad y de comunicación, exportación y PWA publicada |
| Métricas | Las siete definidas por escrito con método de medición | Sí | Sección 4.2.1 |
| | Primeras mediciones de tiempo de aire | Sí | Sección 4.2.1 |
| | Primeras mediciones de tiempo de convergencia (M1) con dos nodos | No | M1 va desde que la computadora entrega la imagen hasta que el último nodo confirma la activación, así que exige la transferencia y la activación, programadas para noviembre. |
| Hardware | Sección de partes y alimentación completada | Sí | Sección 2 |
| | Alimentación independiente de cada nodo armada y verificada | No | Las baterías no están disponibles todavía: cada nodo se alimenta por USB, desde la computadora o desde un cargador. Tampoco se midieron los consumos. Queda para noviembre, con las baterías |

### 4.2.1. Mediciones realizadas

Todas las métricas de la Tabla 1 salen del flujo de eventos que emiten los nodos por la consola, que el panel acumula y exporta. Con lo construido hasta octubre se miden así:

- **M4, latencia de ida y vuelta**: campo `rtt` de cada evento `echo`, en microsegundos desde que empieza a transmitirse el pedido hasta la interrupción de recepción de la respuesta; cada `echo_lost` cuenta como pérdida. Medida en la mesa (Tabla 5) y en campo (Sección 4.2.3).
- **M6, calidad de enlace**: `rssi` y `snr` de los eventos `hello`, `test_rx` y `echo`; la lista `neighbors` del `HELLO` y la respuesta del eco traen cómo escucha el otro nodo, así cada nodo conoce el enlace en los dos sentidos. Medida en campo (Sección 4.2.3).
- **M7, recursos**: reporte del compilador (31 % de la flash en la V4.3) y campo `heap` del evento `status`.
- **Tiempo de aire**: `toa` medido por interrupción y `toa_calc` por la fórmula de Semtech, en cada evento `tx`. Medido en la mesa y en la prueba de capacidad (Sección 4.2.4).
- **Capacidad del canal**: corridas de 300 paquetes seguidos por modulación, con `received × size × 8 / duration` a partir de `run_end` y `run_done` (Sección 4.2.4).
- **M1, M2, M3 y M5** dependen de la transferencia de imágenes y de la VM, programadas para noviembre. El piso de M1 es el tiempo de aire de la imagen más un RTT por ronda de reparación; la PDR de una corrida (`run_end`) anticipa la pérdida del enlace que M3 va a medir sobre fragmentos.

### 4.2.2. Prueba básica en la mesa

Con dos V4.3 una al lado de la otra y los parámetros por defecto se obtuvo:

**Tabla 5.** Mediciones sobre la mesa con dos V4.3.

| Medida | Medido | Calculado |
|---|---|---|
| Tiempo de aire de un `HELLO` de 26 bytes, SF7, 500 kHz, CR 4/5 | 15,669 ms | 15,424 ms |
| RTT de un eco de 16 bytes de cuerpo (trama de 32 bytes) | ≈ 40 ms | 2 × 18,0 ms de aire más procesamiento |
| RSSI entre las placas a +4 dBm | ≈ −35 dBm | — |

La diferencia de unos 245 µs en el tiempo de aire (1,6 %) corresponde a la latencia entre la orden de transmitir por SPI y el comienzo real de la emisión, más la de la interrupción de fin. El RTT no depende de la distancia, sino del SF, del largo del cuerpo y del procesamiento: para 300 m la luz tarda 1 µs.

### 4.2.3. Prueba de campo

La prueba de comunicación en campo se hizo el 11 de octubre sobre la avenida 72 de La Plata. El lugar se eligió como entorno intermedio entre los interiores de la Facultad y el camino semirrural a Punta Lara, donde midió el grupo de 2025 [5]: por el cantero central de la avenida corre la vía del Tren Universitario, que da una línea de vista casi constante entre los dos nodos, pero la atraviesan de forma permanente autos y otros obstáculos.

Un nodo quedó fijo quedó en la esquina de 72 y 1. Otro nodo móvil se detuvo en las esquinas de 72 con 4, 7, 10, 13, 16 y 19. La Figura 5 muestra el recorrido y la Tabla 6 las distancias relativas, calculadas a partir de las coordenadas GPS tomadas en cada esquina.




**Figura 5.** Recorrido de la campaña sobre la avenida 72, desde la esquina de 1 hasta la de 19. Imagen satelital de Google Maps.

**Tabla 6.** Puntos de medición y sus distancias relativas a 1 y 72.

| Esquina de la Av. 72 | Distancia |
|---|---|
| Avenida 1 | 0 m |
| Calle 4 | 452 m |
| Avenida 7 | 856 m |
| Calle 10 | 1290 m |
| Avenida 13 | 1696 m |
| Calle 16 | 2142 m |
| Avenida 19 | 2540 m |

En cada punto el móvil pidió series de ecos de 4 bytes (tramas de 20 bytes), de 100, 50, 25 o 10 ecos según el tiempo disponible, a 915,9 MHz y 500 kHz, con las combinaciones de SF, tasa de código y potencia de la Tabla 9 en el Apéndice A. Cada serie da la PDR de ida y vuelta (respuestas recibidas sobre pedidos), la PDR de ida (pedidos servidos en el nodo fijo), el RSSI y la SNR en los dos sentidos y el RTT.

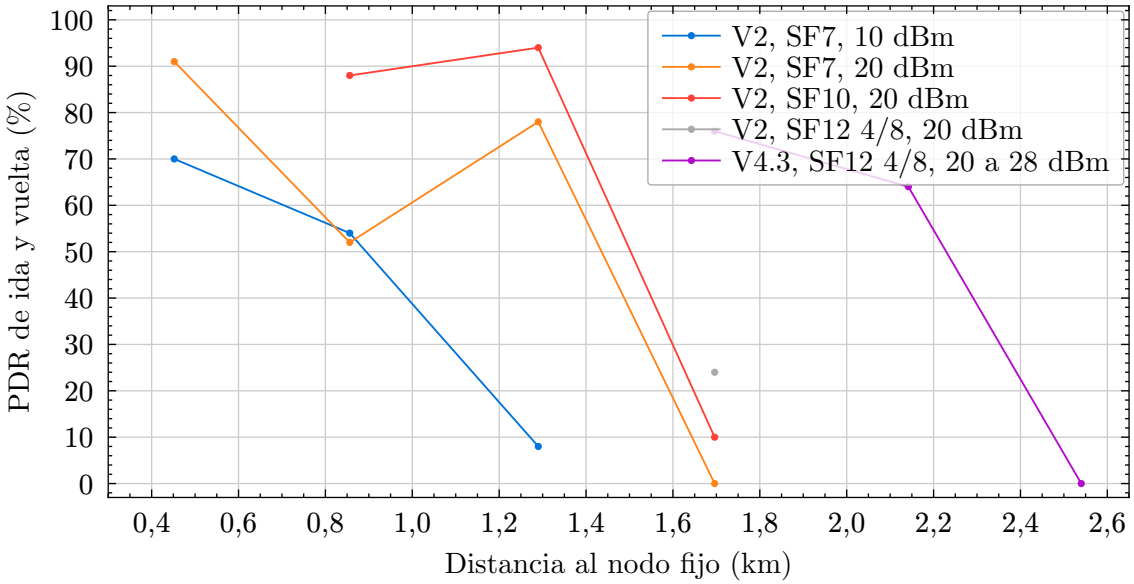



**Figura 6.** PDR de ida y vuelta según la distancia.
La serie de la V4.3 a SF12 usa 20 dBm a 1696 m y 28 dBm a 2142 y 2540 m.

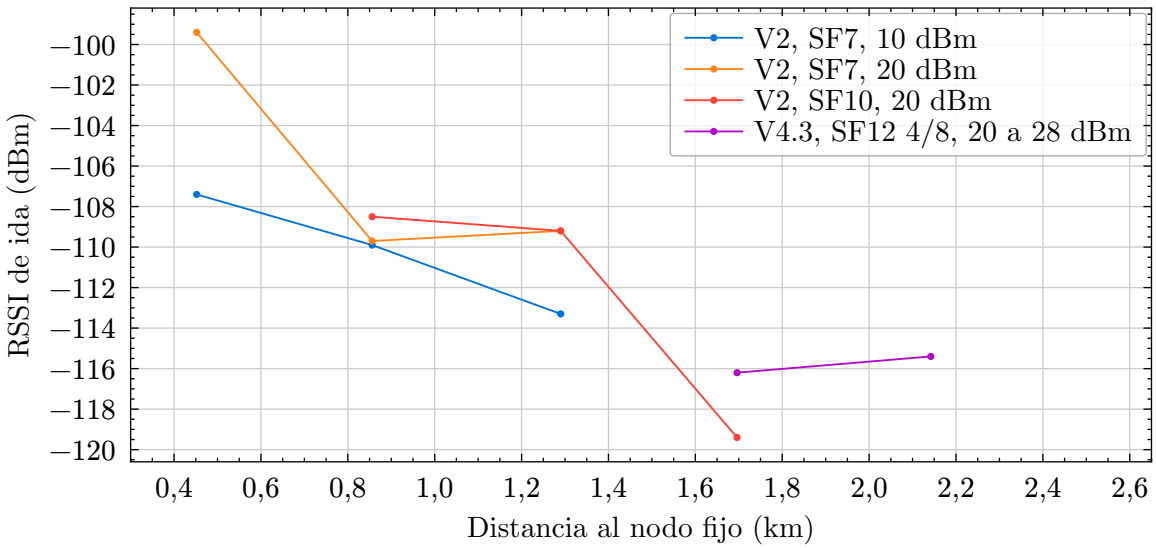



**Figura 7.** RSSI medio de ida (del móvil al nodo fijo) según la distancia.

Con estas mediciones se concluye que:

- **Los enlaces entre modelos funcionan en campo.** Una V2 y una V4.3 intercambiaron ecos en los dos sentidos hasta 1696 m, dos V2 a 452 m con 84 % y dos V4.3 hasta 2142 m con 64 %.
- **Alcance a 500 kHz.** Con la V2 a SF7, el enlace es usable hasta 1290 m (78 % a 20 dBm) y desaparece a 1696 m aun a 20 dBm. SF10 a 20 dBm da 94 % a 1290 m y 10 % a 1696 m. SF12 con CR 4/8 a 20 dBm llega a 1696 m con 24 % desde la V2 y 76 % desde la V4.3. La V4.3 a 28 dBm mantiene 64 % a 2142 m; a 2540 m el nodo fijo recibió 9 de 10 pedidos pero el móvil no recibió ninguna respuesta.
- **Comparación con el informe de 2025.** Aquel grupo, a 125 kHz, obtuvo 50 % a 1250 m con 10 dBm y SF7, y 40 % a 2750 m con 20 dBm y SF12. Nosotros, a 500 kHz y en una avenida urbana, obtuvimos 8 % a 1290 m con 10 dBm y SF7 y 78 % con 20 dBm. Pero con la V4.3 a 28 dBm y SF12, el alcance vuelve al orden de los 2,1 a 2,5 km de aquel informe. Se cree que esto es principalmente debido al incremento de ancho de banda, mas no se puede cambiar por limitantes del ENACOM.
- **Tasa de código.** A 856 m con SF7 y 20 dBm, pasar de CR 4/5 a 4/8 subió la PDR de 52 % a 92 %. A 1696 m con SF10 no cambió nada: 10 % con las dos.
- **Latencia (M4).** El RTT medio fue de 31,4 ms a SF7, 191,3 ms a SF10 y 873,5 ms a SF12 con CR 4/8, contra dos tiempos de aire de 28,3, 185,3 y 856,1 ms: el procesamiento de un eco cuesta entre 3 y 17 ms, más a mayor SF.
- **Asimetría en el extremo móvil.** A 2142 y 2540 m, la SNR de vuelta fue entre 4 y 6 dB peor que la de ida con RSSI parecidos. A igual señal, peor SNR significa más ruido en el receptor móvil, que estaba junto a una computadora y al tránsito de la avenida. Explica que a 2540 m el fijo oyera 9 de 10 pedidos y el móvil ninguna respuesta.

El barrido de potencia con tramas largas en un punto fijo, que el plan preveía como prueba aparte, no se repitió. La caracterización del alcance según la potencia y el SF con estas mismas placas ya la hicieron los estudiantes del año pasado sobre el camino a Punta Lara [5], y el mal tiempo de los días previos a la entrega dejó una sola salida posible, que se destinó a la avenida 72. La escalera de potencia con ecos a tres distancias de la Tabla 9 cubre lo que ese barrido habría medido, salvo el efecto del largo de la trama, que queda cubierto por la prueba de capacidad.

### 4.2.4. Prueba de capacidad

La prueba de capacidad se hizo con `EF35` (V2) y `D6D8` (V4.3) conectadas por USB a la misma computadora y a un metro una de otra: el receptor veía entre −27 y −35 dBm. Parámetros: 915,9 MHz, 500 kHz, CR 4/5, cuerpos de 239 bytes (tramas de 255), 300 paquetes seguidos por punto y SF7 a SF12, con la V2 a +2 dBm y la V4.3 a +4 dBm con el LNA puenteado. El panel recorrió los seis puntos en 21 min por dirección, primero con la V2 como emisor y después con la V4.3. Las medidas se detallan en la Tabla 10 del Apéndice A.

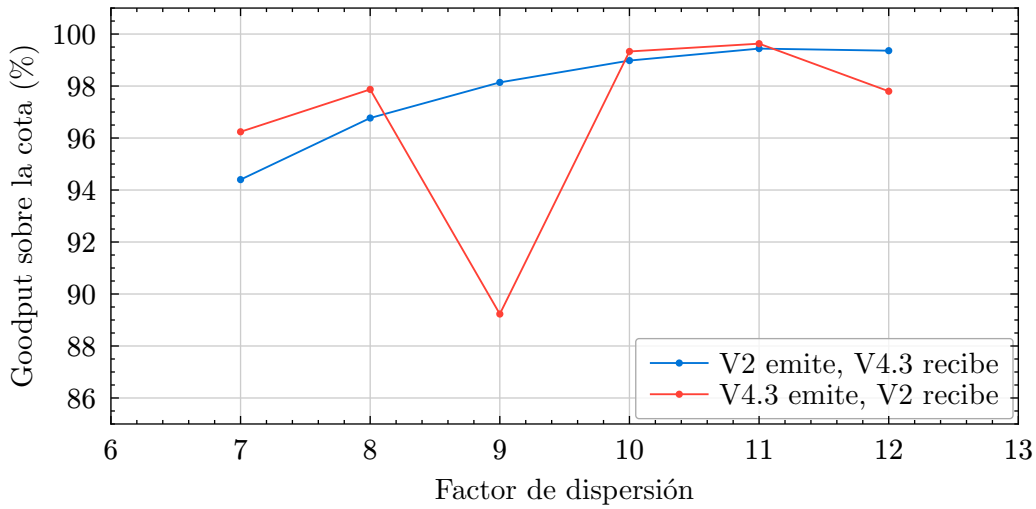



**Figura 8.** Goodput medido como fracción de la cota de Semtech, según el SF y el sentido.

Se concluye:

- **Capacidad del canal.** Con SF7 y 500 kHz el goodput sostenido fue de 18,1 kbit/s con la V2 como emisor y 18,4 kbit/s con la V4.3, con 300 de 300 paquetes recibidos en los dos sentidos: PDR ≥ 99 % con 95 % de confianza. Es la capacidad del canal con la modulación por defecto, el 94 y el 96 % de la cota de Semtech de 19,1 kbit/s. De SF8 en adelante la relación sube hasta el 99 %, porque la separación entre tramas pesa cada vez menos frente al tiempo de aire.
- **Separación entre tramas.** Es constante con el SF: 5,9 ms cuando emite la V2 y 3,6 ms cuando emite la V4.3. Es lo que cuesta la interrupción de fin, una vuelta del bucle y cargar los 255 bytes de la trama siguiente por SPI; la diferencia entre modelos sale de la velocidad del microcontrolador y del manejo del FIFO de cada transceptor.
- **Pérdidas sobre la mesa.** Nueve de los doce puntos entregaron el 100 %. Los otros tres perdieron paquetes a −30 dBm, 85 dB por encima de la sensibilidad, así que no es presupuesto de enlace: la V4.3 perdió el primer paquete de SF12, recién reconfigurada; la V2 perdió 29 paquetes repartidos en SF9, sin errores de CRC, y 6 en SF12, cinco de ellos con error de CRC y con el RSSI más alto de toda la prueba, −25,8 dBm. Lo segundo apunta a la saturación del SX1276 con las placas demasiado cerca; lo de SF9 no tiene explicación todavía. Los tres puntos se repiten con las placas a tres metros, que son dos minutos de prueba.

## 4.3. Documentación del código

El código está en el repositorio de la cátedra (Sección 5.3). El firmware vive en `firmware/`, como sketch de Arduino con el código agrupado por responsabilidad. El panel de control se encuentra en `control/`. La Tabla 7 lista los módulos del firmware. Los contratos entre las partes están escritos aparte, en `docs/protocol/`: los parámetros de radio, la trama y la consola.

**Tabla 7.** Módulos del firmware.

| Carpeta | Módulo | Responsabilidad |
|---|---|---|
| `application/` | `node_application` | Bucle principal, comandos de consola (SimpleCLI), emisión de eventos y cola de transmisión |
| `board/` | `board_support` | Modelo de placa, alimentación (Vext), etapa de RF de la V4.3 y traducción de la potencia |
| `radio/` | `radio_link` | Transceptor LoRa: parámetros en caliente, transmisión y recepción con marcas de tiempo |
| | `radio_settings`, `time_on_air` | Parámetros de radio y su validación; fórmula de Semtech del tiempo de aire |
| `network/` | `frame`, `node_identity` | Formato de la trama y etiqueta HMAC-SHA256; identificador, época y secuencia |
| | `duplicate_filter`, `neighbor_table` | Descarte de repetidos; tabla de vecinos |
| `services/` | `presence_service`, `echo_service`, `test_run_service` | Calendario y cuerpo del `HELLO`; eco pendiente y plazos; corridas del emisor y estadísticas del receptor |
| `console/` | `console`, `event_fields` | Consola y eventos JSON (ArduinoJson) |
| | `serial_transport`, `wireless_transport` | Transportes de la consola: serie y WebSocket sobre el punto de acceso |
| `display/` | `debug_display` | OLED: fila de estado y registro de depuración (U8g2) |

Todo el firmware corre en el bucle principal de Arduino, en el orden fijo de la Código 1: primero la radio, después la cola de transmisión, la consola, los temporizadores y por último el OLED. Ninguna función bloquea, salvo el envío del cuadro al OLED (unos 30 ms), que por eso va último y nunca con la radio ocupada. La única tarea que se suma es la del servidor HTTP de ESP-IDF, que solo mueve líneas de consola.

```
void NodeApplication::poll() {
  const uint32_t now = millis();
  serviceRadio(now);
  serviceTransmitter(now);
  serviceConsole();
  serviceTimers(now);
  // Sending a frame to the display blocks for about 30 ms. Never while the radio is transmitting,
  // has frames waiting or expects an echo reply, so it returns to reception in time.
  if (!radio_.isTransmitting() && queueCount_ == 0 && !echo_.isPending()) {
    display_.poll(now);
  }
}
```

**Código 1.** Bucle principal del firmware (`NodeApplication::poll()` en `firmware/src/application/node_application.cpp`).

### 4.3.1. Código que maneja el hardware

Cada módulo del firmware es una clase con `begin()` y un método que se atiende desde el bucle. Para armar un nodo desde cero, el orden de arranque es el de `NodeApplication::begin()`: primero la placa, después la consola serie, el OLED, la identidad, la radio y por último el punto de acceso WiFi.

- **Placa y etapa de RF** (`BoardSupport`, en `board/`). `begin()` enciende Vext, que alimenta el OLED, y en la V4.3 alimenta y habilita la etapa KCT8103L verificando que responda como tal; tiene que correr antes que la radio y el OLED. `model()` dice qué placa es, decidido en tiempo de compilación por la placa elegida. `transceiverPowerFor(dBm)` traduce la potencia pedida en el conector de antena al ajuste del transceptor, restando la ganancia de la etapa en la V4.3, e `isTransmitPowerReachable(dBm)` valida el rango de cada modelo.

- **Transceptor LoRa** (`RadioLink`, en `radio/`). `begin(placa, parámetros)` abre el SPI, configura el SX1276 o el SX1262 con RadioLib [6], sube el límite de corriente a 140 mA, en la V4.3 deja el TCXO siempre encendido y carga la tabla del conmutador de RF, instala la interrupción y deja el transceptor en recepción continua. `startTransmit(datos, largo)` envía una trama y guarda la marca de tiempo de inicio; devuelve un código propio si hay una recepción pendiente de leer, en cuyo caso el bucle reintenta en la vuelta siguiente. `poll(trama)` se llama en cada vuelta y devuelve nada, fin de transmisión o trama recibida; en el último caso llena la trama con los bytes, el RSSI corregido, la SNR, el error de frecuencia y la marca de tiempo de la interrupción. `applySettings(parámetros)` cambia la modulación o la potencia en caliente y vuelve a recepción; `measuredTimeOnAirMicros()` da el tiempo de aire medido de la última transmisión. La interrupción de la radio solo guarda la marca de tiempo y una bandera: todo el trabajo ocurre en `poll()`.
- **OLED** (`DebugDisplay`, en `display/`). `begin()` inicializa el SSD1306 o SSD1315 con U8g2 [7]; `setStatusLine(texto)` fija la fila superior y `appendLine(texto)` agrega una línea al registro de cinco filas. `poll(ahora)` redibuja solo si algo cambió, como mucho cada 200 ms, y `setSuspended(true)` lo detiene durante las corridas, porque enviar un cuadro bloquea unos 30 ms.

### 4.3.2. Trama y protocolo de enlace

Cada trama LoRa lleva una cabecera de 8 bytes (tipo, origen, destino, época y secuencia), un cuerpo de hasta 239 bytes y una etiqueta de 8 bytes: los primeros 8 bytes de HMAC-SHA256 con la clave de red, calculado sobre la versión del protocolo, la cabecera y el cuerpo. La versión no viaja: una trama armada con otra versión o con otra clave falla la verificación y se descarta como tráfico ajeno. El identificador del nodo son los dos últimos bytes de su MAC; la época es un contador guardado en NVS que aumenta en cada arranque y cada vez que la secuencia de 16 bits da la vuelta.

Los módulos de `network/` se usan así: `NodeIdentity::begin()` deriva el identificador y avanza la época; `assignSequence()` da la época y la secuencia de cada trama nueva. `encodeFrame(cabecera, cuerpo, largo, salida)` escribe la trama completa con su etiqueta y `decodeFrame(datos, largo, …)` verifica la etiqueta y separa cabecera y cuerpo. `DuplicateFilter::acceptIfNew(origen, época, secuencia)` responde si la trama es nueva y la recuerda; `NeighborTable::recordFrame()` actualiza la calidad de enlace de un vecino con cualquier trama válida, `recordHello()` guarda lo que el vecino anuncia y `expire()` borra a los que llevan 60 s sin oírse. Los cuerpos de cada tipo de mensaje los codifican y decodifican los servicios de `services/`: `PresenceService` el `HELLO`, `EchoService` el pedido y la respuesta del eco y `TestRunService` las tramas de una corrida.

Hay cuatro tipos de mensaje: `HELLO` (difusión cada 10 s ± 2 s, con el modelo, la versión del programa, el contador de transmisiones y la tabla de vecinos con RSSI y SNR), `ECHO_REQUEST` y `ECHO_REPLY` (la respuesta tiene el mismo largo que el pedido y lleva el RSSI y la SNR con que llegó) y `TEST` (paquetes numerados de una corrida). El formato exacto está en `docs/protocol/frame.md`.

### 4.3.3. Código que maneja las comunicaciones con la computadora

La consola es la misma por los dos transportes (`Console`, en `console/`): cada evento es un objeto JSON en una línea, armado con ArduinoJson [8], y cualquier otra línea es texto de depuración. Para emitir un evento, `beginEvent(nombre)` devuelve un objeto JSON con los campos comunes `t` y `ev`, el llamador agrega los suyos y `writeEvent()` lo serializa y lo escribe por la serie y por todos los clientes WebSocket; `writeDebug(texto)` escribe una línea de texto. `readCommand(buffer)` entrega la próxima línea de comando que haya llegado por cualquiera de los dos transportes. Si el búfer de salida de un transporte está lleno, la línea se descarta en ese transporte y se cuenta: la consola nunca demora a la radio.

- **Serie USB** (`SerialTransport`): `begin(921600)` abre el puerto con un búfer de transmisión de 4096 bytes; `readLine()` arma la línea de a un carácter y descarta las que superan los 200; `writeLine()` nunca bloquea.
- **WiFi** (`WirelessTransport`): `begin(ssid, clave, canal, clientes)` levanta el punto de acceso `J5-<id>` con WPA2 y un WebSocket en `ws://192.168.4.1/console` sobre `esp_http_server`, el servidor de ESP-IDF, en una tarea fijada al núcleo 0. Esa tarea solo copia líneas: las que llegan van a una cola de 8 comandos que el bucle vacía con `readLine()`, y las que salen las entrega `writeLine()` con un tope de 32 en vuelo. `end()` apaga el punto de acceso. Un cliente cuenta como conectado desde su primer mensaje, porque ESP-IDF 5.5.5 completa el handshake sin avisar al manejador (Sección 4.4).

- **Comandos**: el bucle lee hasta dos líneas por vuelta y las interpreta con SimpleCLI [9], que separa las palabras y valida los argumentos posicionales. Los comandos son `help`, `status`, `radio`, `echo`, `run`, `wifi` y `resend`; cada error produce un evento `error` con el motivo (`usage`, `out_of_range`, `busy`, etcétera). La Tabla 8 resume los eventos que emite el nodo; el contrato completo está en `docs/protocol/console.md`.

**Tabla 8.** Eventos de la consola.

| Evento | Cuándo y qué informa |
|---|---|
| `boot`, `radio`, `wifi` | Al arrancar: identificador, modelo, época y versión; parámetros de radio; estado del punto de acceso |
| `status`, `neighbor` | Con el comando `status`: contadores, memoria libre y un `neighbor` por vecino con RSSI, SNR y antigüedad |
| `tx` | Al terminar cada transmisión: tipo, destino, secuencia, largo, tiempo de aire medido y calculado |
| `hello` | Al recibir un `HELLO`: emisor, calidad y la lista de vecinos del emisor |
| `echo_served`, `echo`, `echo_lost` | Eco respondido; respuesta recibida con `rtt` y calidad de ida y vuelta; plazo vencido |
| `run_start`, `run_done`, `test_rx`, `run_end` | Corrida del emisor (inicio, fin con duración) y del receptor (cada paquete; resumen con PDR, RSSI medio, mínimo y máximo y SNR media) |
| `duplicate`, `drop`, `id_conflict`, `error` | Trama repetida; trama inválida con el motivo (`crc`, `short`, `tag`, `type`, `body`); otro nodo con el mismo identificador; comando rechazado o falla de la radio |

### 4.3.4. Interfaz web y software en la computadora

El panel de control es una aplicación de una sola página escrita en TypeScript con Svelte 5 [10] para la interfaz y Tailwind CSS 4 [11] para los estilos, compilada con Vite+ [12] y publicada como PWA con vite-plugin-pwa y Workbox. El Código 2 muestra su estructura: la interfaz de usuario está en `components/`, con un componente por pestaña, y en `state/`, que es el estado reactivo que los componentes presentan; la comunicación con la placa está en `transports/`, con Web Serial [13] y el WebSocket detrás de una misma interfaz, en `console/`, que interpreta cada línea contra el contrato de eventos escrito como esquemas de Zod [14], y en `nodes/`, que une un transporte con el estado derivado de sus eventos. El *service worker* guarda la página al instalarse, así el panel abre aunque la computadora esté asociada al punto de acceso de un nodo y sin internet.

```
control/src/
├── main.ts                 punto de entrada: monta App.svelte
├── App.svelte              raíz de la interfaz, con las pestañas
├── app.css                 estilos globales y Tailwind
├── service-worker.ts       funcionamiento sin conexión (Workbox)
└── lib/
    ├── components/         componentes Svelte; sections/ tiene uno por pestaña
    ├── state/              estado reactivo de la aplicación
    ├── console/            eventos (Zod), comandos y lectura de líneas de la consola
    ├── transports/         Web Serial y WebSocket detrás de una misma interfaz
    ├── nodes/              conexión y estado derivado de cada nodo
    ├── radio/              parámetros de radio y tiempo de aire
    ├── measurements/       corridas, pruebas de capacidad y de comunicación, eco y exportación
    ├── utils/              formato de valores y utilidades
    └── testing/            sesiones grabadas y nodo simulado para las pruebas
```

**Código 2.** Estructura del panel de control.

Los comandos para trabajar con el panel, desde `control/`, son los de Vite+:

```
vp install        # instala las dependencias
vp dev            # servidor de desarrollo en http://localhost:5173
vp build          # compila la versión de producción en dist/
vp preview        # sirve dist/ para probarla
vp test           # corre las 159 pruebas
vp check          # revisa formato, lint y tipos
vp run typecheck  # revisa los tipos de los componentes Svelte y del service worker
vp run deploy     # compila y publica en Cloudflare
```

### 4.4. Problemas encontrados y soluciones

1. **Regulación de la banda de 915 MHz.** La Res. ENACOM 4653/2019 [3] habilita la banda de 915 a 928 MHz sin autorización individual solo para modulación digital de banda ancha (ancho de al menos 500 kHz, hasta 1 W), salto de frecuencia o sistemas de hasta 200 µW de PIRE. LoRa en un canal fijo de 125 o 250 kHz cae en la última categoría, por debajo de la potencia mínima de las dos placas. **Solución**: el sistema y todas las mediciones usan canales de 500 kHz, que entran como banda ancha, con un tope de unos 26 dBm; los canales de 125 y 250 kHz quedaron fuera del alcance del proyecto, aunque el firmware los conserva como parámetro (ADR 0004). El firmware no impone aún estos límites; hacerlo es trabajo futuro.
2. **Riesgo de dañar el SX1262 con el LNA.** La etapa de RF de la V4.3 tiene un LNA de 21 dB; con las placas sobre la mesa la señal recibida supera el nivel que satura el transceptor. **Solución**: la V4.3 arranca con el LNA puenteado y el panel no permite cambiarlo; se activa a mano desde la consola solo para las pruebas a distancia.
3. **Dos placas con transceptores y rangos de potencia distintos.** El SX1276 de la V2 y el SX1262 de la V4.3 no comparten SF5 y SF6, y la V4.3 entrega de 4 a 28 dBm con una ganancia que depende del nivel. **Solución**: la potencia se expresa siempre en el conector de antena y cada placa la traduce; los parámetros por defecto se eligieron en el rango común (SF7 y +4 dBm) y las pruebas se hacen por par de modelos y en ambos sentidos.
4. **Consumo de la V4.3.** El plan estimaba 120 mA para el transceptor transmitiendo; la hoja de datos de la V4.3 indica 750 mA a 27 dBm por USB [15], más de lo que entrega un puerto USB 2.0. **Solución**: fuente USB de al menos 1 A por nodo y potencia mínima sobre la mesa. La medición real se realizará en el futuro con las baterías.
5. **El panel por WiFi enviaba comandos pero no recibía nada. Causa**: ESP-IDF 5.5.5, la base del core de Arduino 3.3.12, completa el handshake del WebSocket sin llamar al manejador, así que el cliente nunca quedaba registrado. **Solución**: el cliente se registra con su primer mensaje y el panel envía `status` apenas abre la conexión.
6. **Polaridad de Vext en la V4.3.** El esquemático y el texto de la hoja de datos se contradicen sobre el nivel que enciende Vext.

# 5. Documentación relacionada

## 5.1. Fotos de las partes construidas



**Figura 9.** Fotografía desde Av. 1.

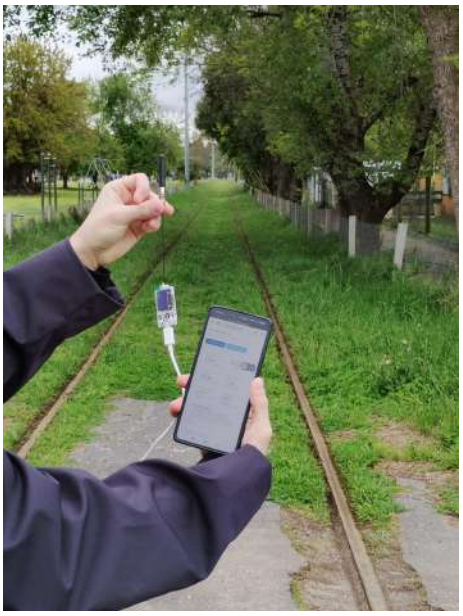


**Figura 10.** Fotografía desde C. 4.

## 5.2. Capturas del panel de control

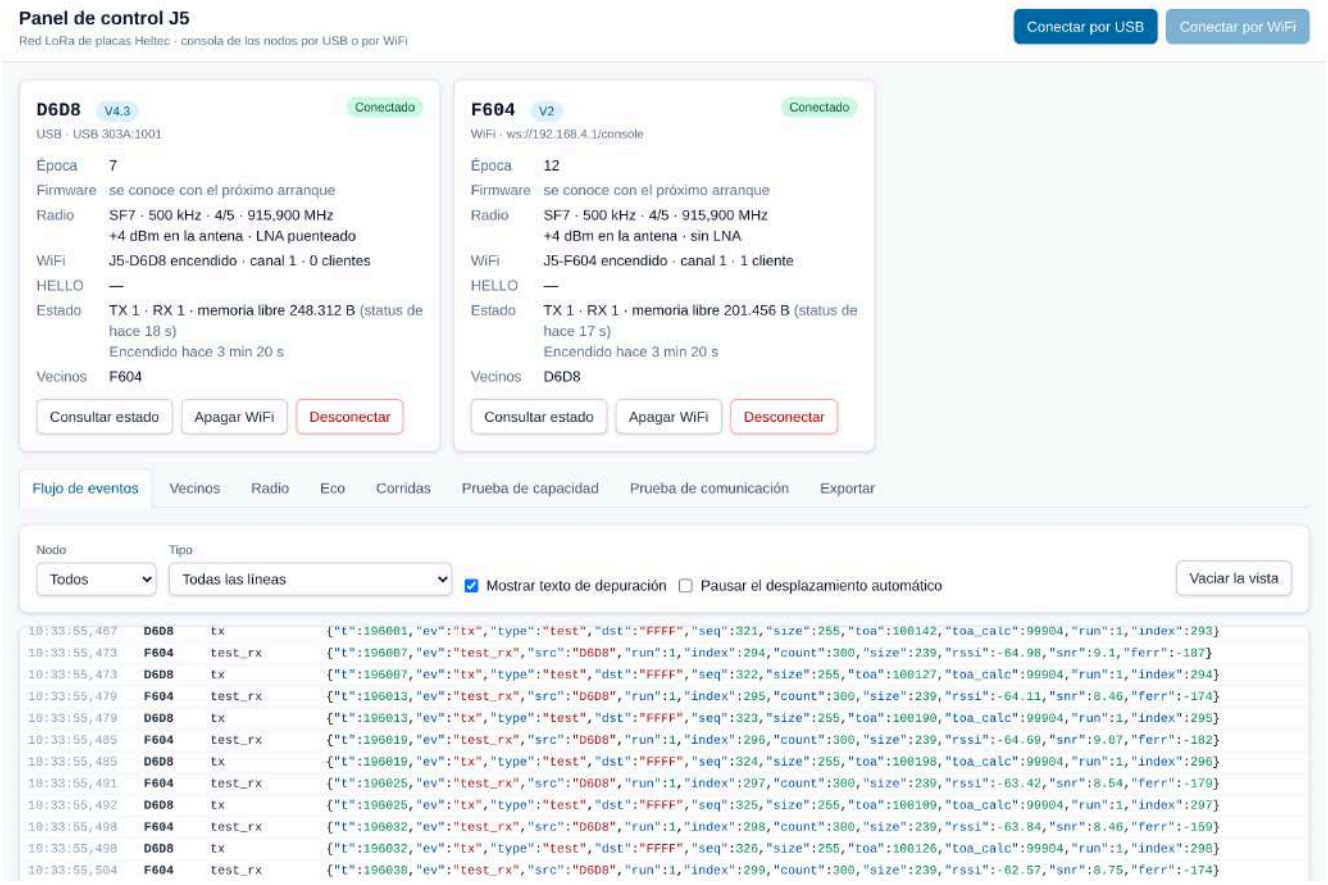



**Figura 11.** Flujo de eventos con dos nodos conectados, uno por USB y otro por WiFi. Corresponde a la consola de la Sección 4.3.3.

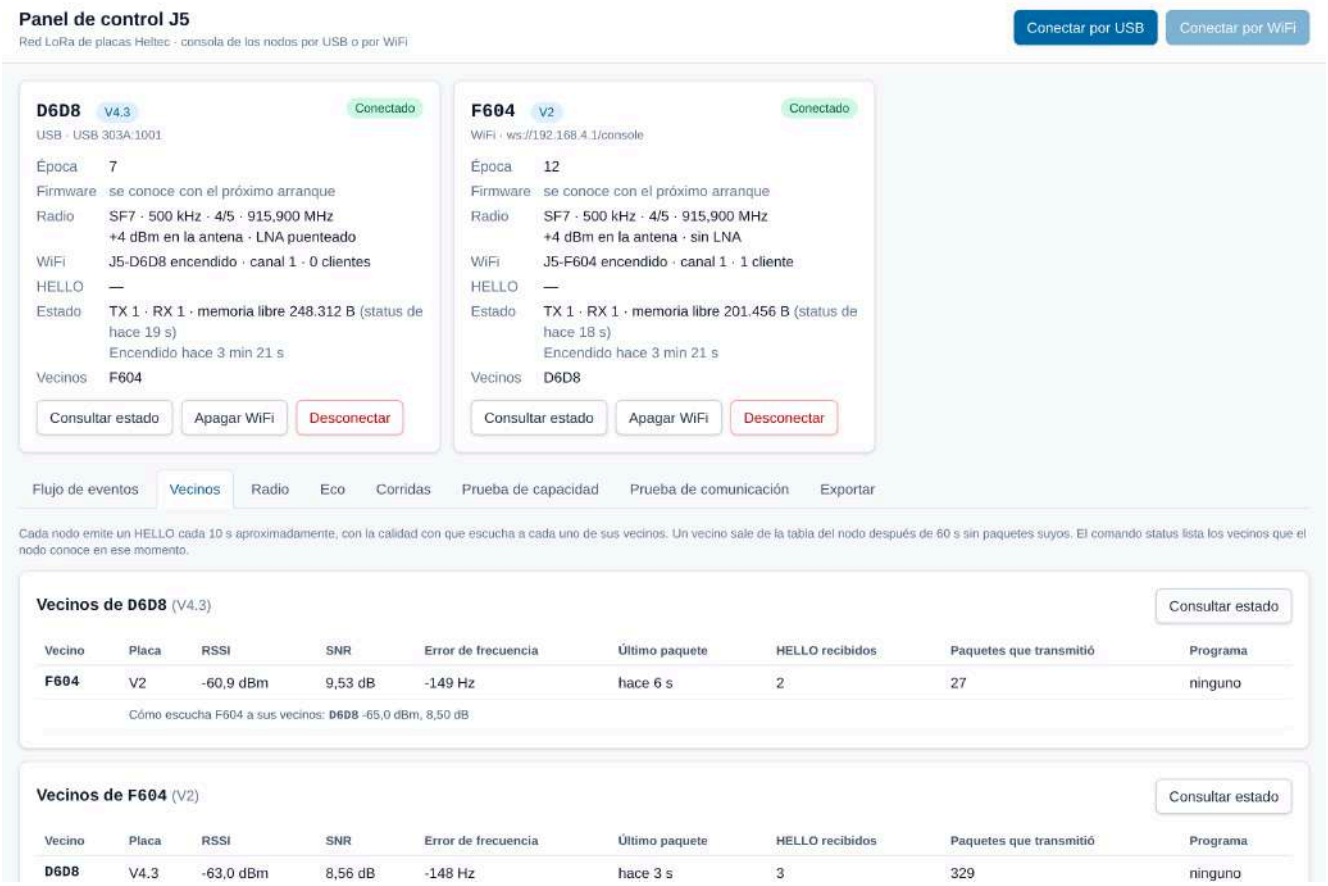



**Figura 12.** Vecinos: lo que cada nodo sabe de los demás a partir de los `HELLO`, con la calidad en los dos sentidos (M6).

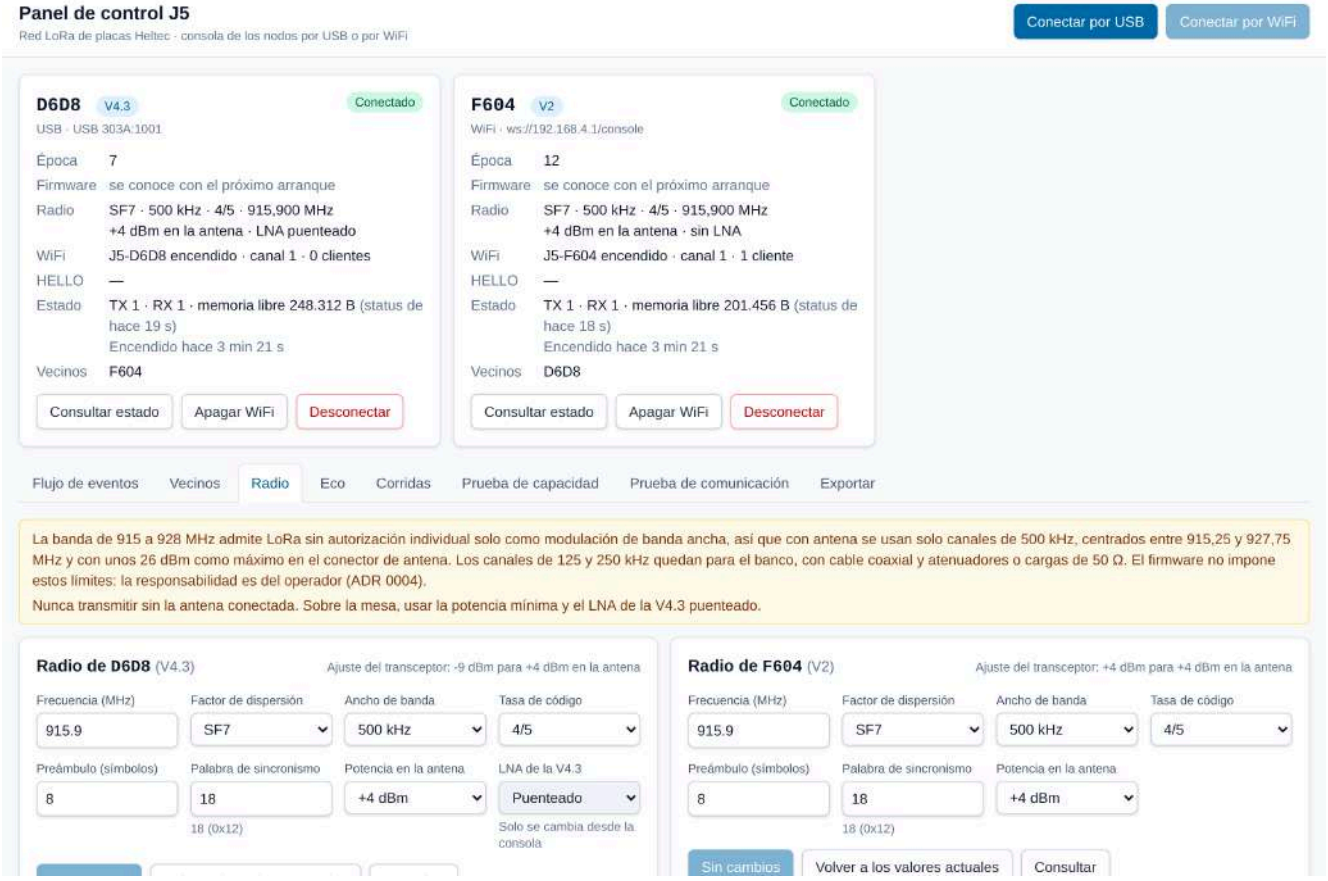



**Figura 13.** Parámetros de radio de cada nodo, validados según el modelo de placa.

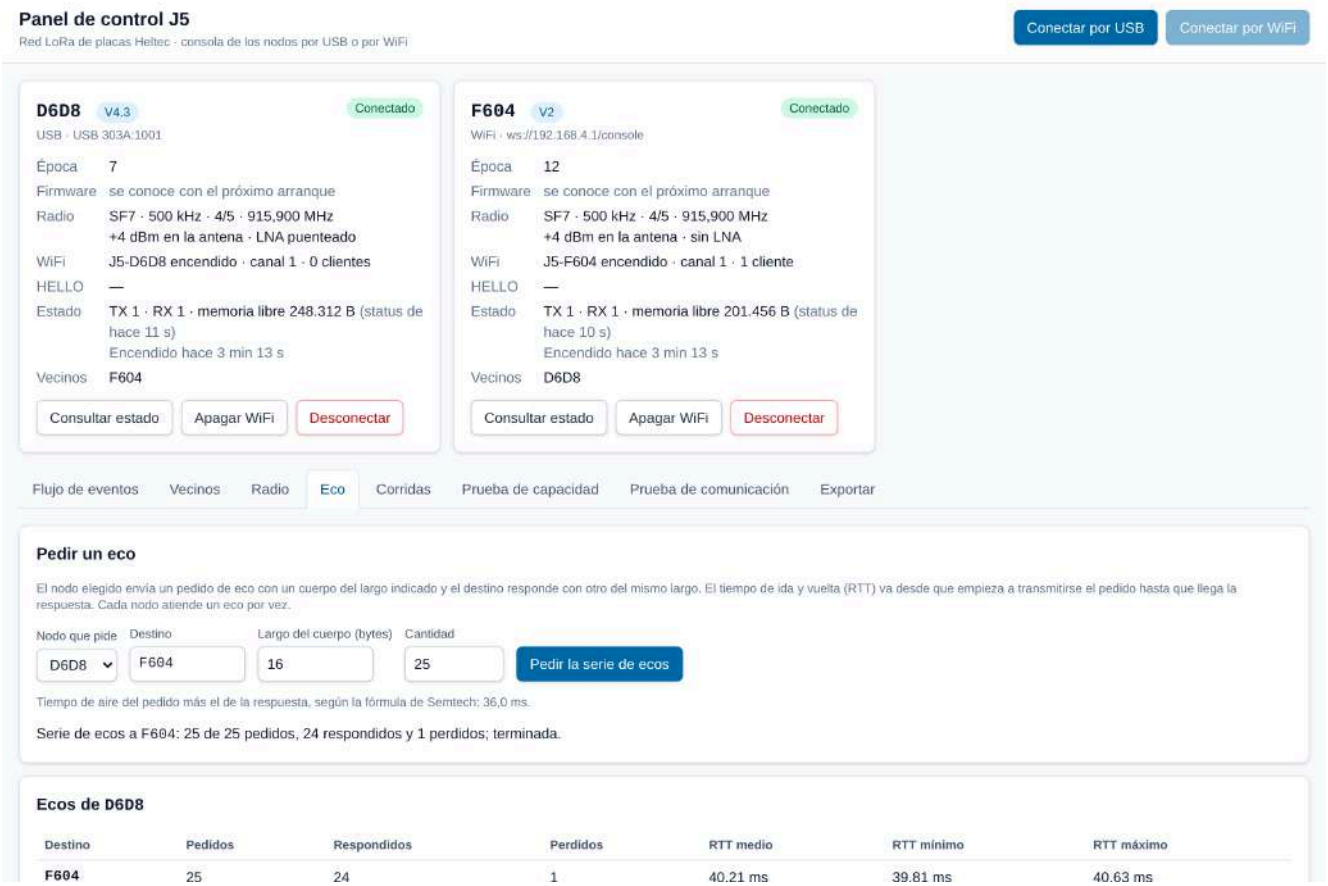



**Figura 14.** Eco: serie de ecos con el RTT (M4) y la calidad de ida y de vuelta.

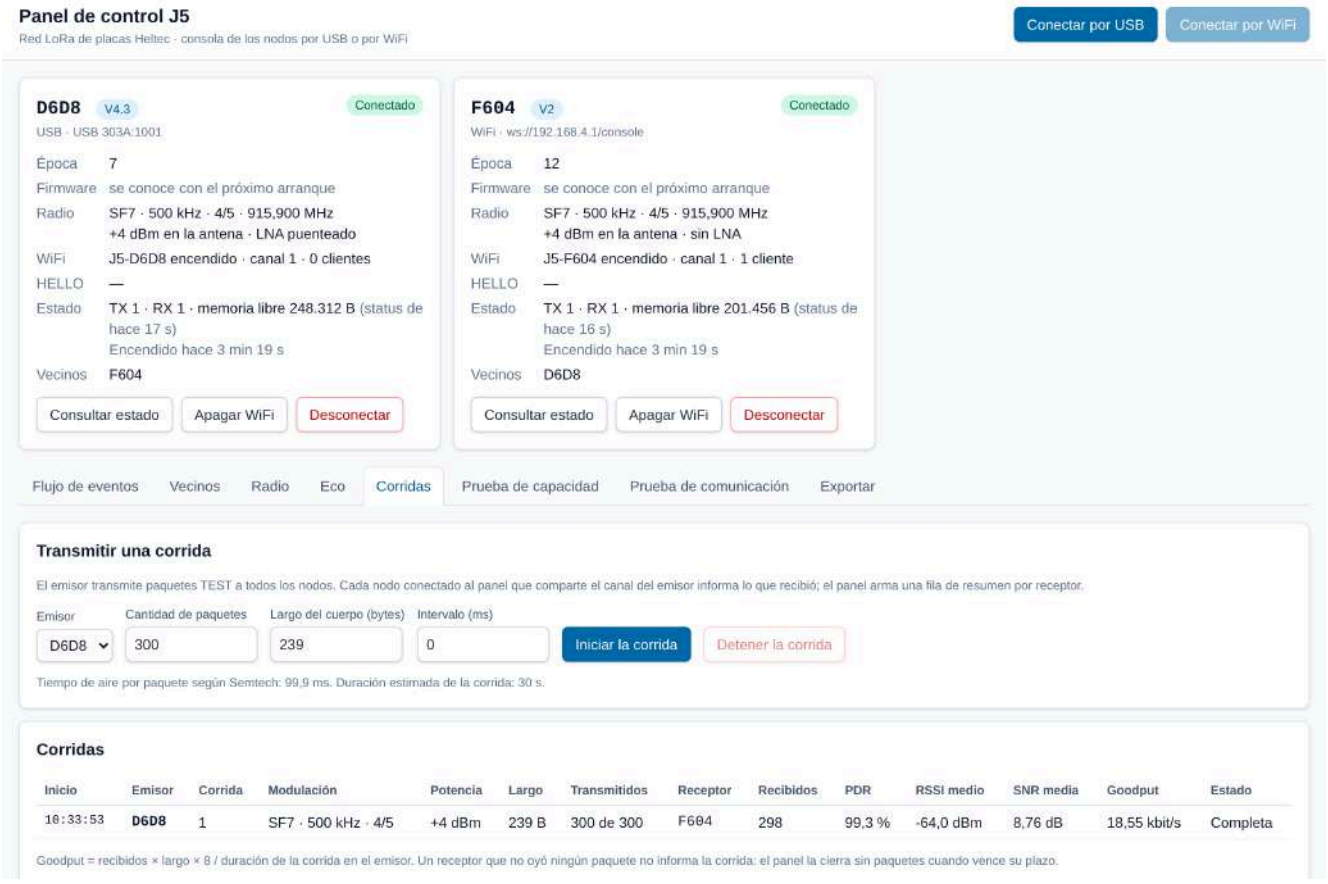



**Figura 15.** Corridas: resumen de las corridas con PDR y RSSI por receptor.

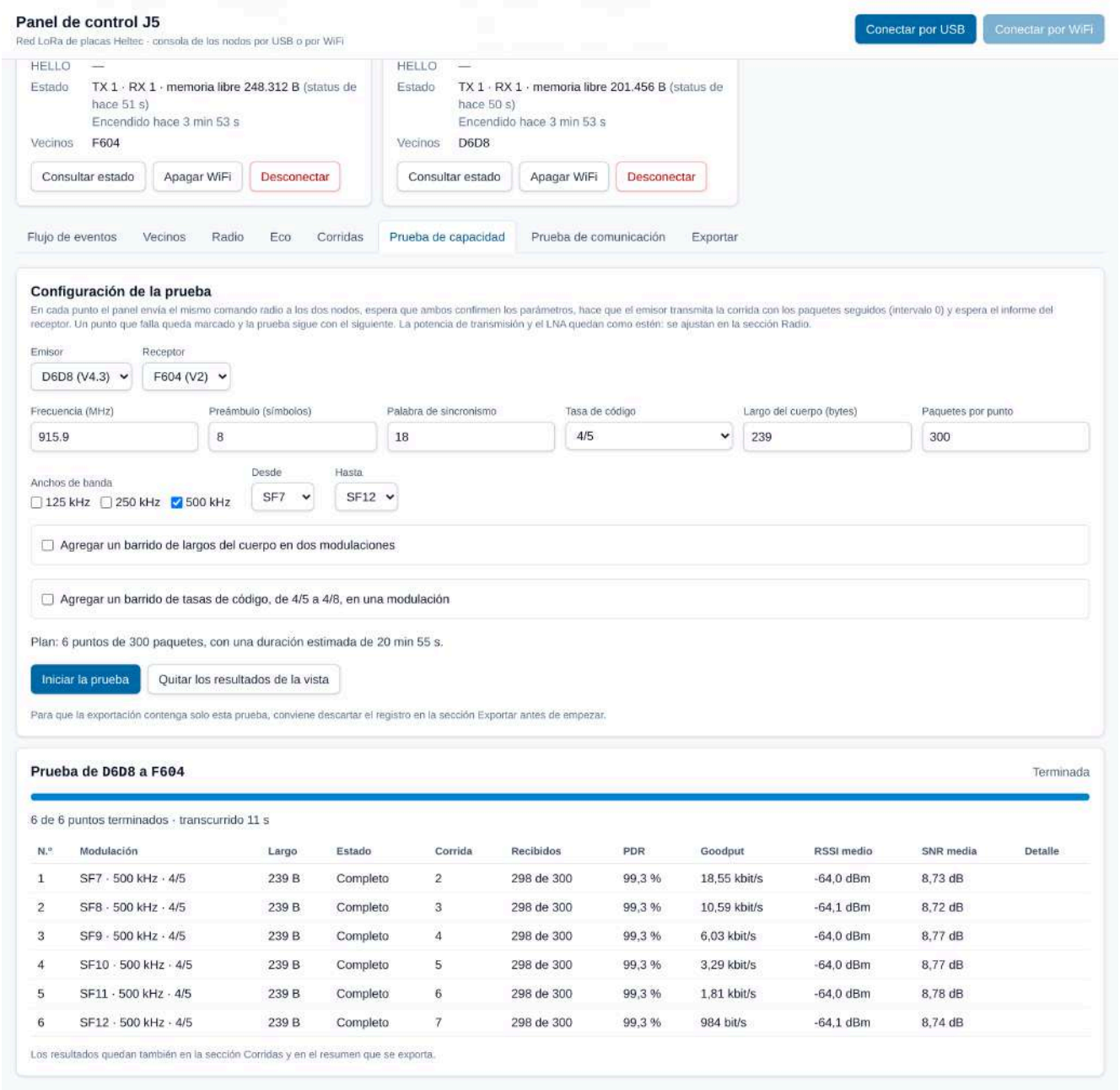



**Figura 16.** Prueba de capacidad: matriz de modulaciones con el *goodput* y la PDR de cada punto.

## 5.3. Enlaces

- **Repositorio de código fuente**: github.com/tpII/2026-j5-lora-ota-update con el firmware, el panel, los contratos y las ADR.
- **Panel de control publicado**: https://j5-lora-ota.finisterre.app
- **Bitácora**: https://jms.ar/tdp2-bitacora
- **Video**: https://jms.ar/tdp2-e2-video

Las bibliotecas que usa el firmware se instalan por versión desde `firmware/sketch.yaml` con arduino-cli: el core `esp32:esp32` 3.3.12 de Espressif [4], RadioLib 7.8.1 [6], ArduinoJson 7.4.3 [8], SimpleCLI 1.1.4 [9] y U8g2 2.36.19 [7]. Desde el IDE de Arduino 2.3 hay que instalarlas a mano con esas mismas versiones. Las del panel están fijadas en `control/package.json` y `pnpm-lock.yaml`: Svelte 5, Tailwind CSS 4, Zod 4, vite-plugin-pwa 2, Workbox 7, Vite+ 1.0 y Wrangler 4.

# Bibliografía

[1] J. W. Hui y D. Culler, «The Dynamic Behavior of a Data Dissemination Protocol for Network Programming at Scale», en *Proceedings of the 2nd International Conference on Embedded Networked Sensor Systems (SenSys '04)*, ACM, 2004, pp. 81-94. doi: 10.1145/1031495.1031506.
[2] P. Levis, N. Patel, D. Culler, y S. Shenker, «Trickle: A Self-Regulating Algorithm for Code Propagation and Maintenance in Wireless Sensor Networks», en *Proceedings of the 1st Symposium on Networked Systems Design and Implementation (NSDI '04)*, USENIX, 2004.
[3] Ente Nacional de Comunicaciones (ENACOM), «Resolución 4653/2019: uso de la banda de 915 a 928 MHz por sistemas de espectro ensanchado y modulación digital de banda ancha, anexo IF-2019-39048876-APN-DNPYC#ENACOM». [En línea]. Disponible en: https://www.boletinoficial.gob.ar/
[4] Espressif Systems, «Arduino core for the ESP32, versión 3.3.12». [En línea]. Disponible en: https://github.com/espressif/arduino-esp32
[5] B. Daskus, T. Simonte, y V. Franco, «Taller de Proyecto II, informe final: F.1 LoRa», technical report, dic. 2025.
[6] J. Gromeš, «RadioLib: Universal wireless communication library for embedded devices». [En línea]. Disponible en: https://github.com/jgromes/RadioLib
[7] O. Kraus, «U8g2: monochrome graphics library for embedded devices, versión 2.36.19». [En línea]. Disponible en: https://github.com/olikraus/u8g2
[8] B. Blanchon, «ArduinoJson, versión 7.4.3». [En línea]. Disponible en: https://arduinojson.org/
[9] S. Kremser, «SimpleCLI, versión 1.1.4». [En línea]. Disponible en: https://github.com/SpacehuhnTech/SimpleCLI
[10] Svelte contributors, «Svelte 5». [En línea]. Disponible en: https://svelte.dev/
[11] Tailwind Labs, «Tailwind CSS 4». [En línea]. Disponible en: https://tailwindcss.com/
[12] VoidZero Inc., «Vite+ 1.0». [En línea]. Disponible en: https://viteplus.dev/
[13] Web Platform Incubator Community Group, «Web Serial API». [En línea]. Disponible en: https://wicg.github.io/serial/
[14] C. McDonnell, «Zod 4: TypeScript-first schema validation». [En línea]. Disponible en: https://zod.dev/
[15] Heltec Automation, «WiFi LoRa 32 V4.3.1 Datasheet, Rev. 1.4». 2025. [En línea]. Disponible en: https://s.heltec.cn/download/WiFi_LoRa_32_V4/datasheet/WiFi_LoRa_32_V4.3.1_Datasheet.pdf

# A. Datos de las mediciones

## A.1. Prueba de campo de octubre

**Tabla 9.** Series de ecos de la campaña de campo, en orden cronológico. PDR de ida y vuelta: respuestas sobre pedidos; «servidos»: pedidos que llegaron al nodo fijo. RSSI en dBm y SNR en dB, promedio de la serie, ida y vuelta.

| m | Emisor | SF · CR | dBm | Resp./ped. | PDR | Servidos | RSSI ida / vuelta | SNR ida / vuelta | RTT (ms) |
|---|---|---|---|---|---|---|---|---|---|
| 452 | F8CD (V2) | SF7 · 4/5 | 20 | 0/26 | 0 % | — | — / — | — / — | — |
| 452 | F8CD (V2) | SF7 · 4/5 | 20 | 84/100 | 84 % | 90 | -110,1 / -107,5 | -1,34 / -0,68 | 33,0 |
| 452 | F8CD (V2) | SF7 · 4/5 | 10 | 70/100 | 70 % | — | -107,4 / -95,8 | -1,83 / 4,55 | 31,4 |
| 452 | F8CD (V2) | SF7 · 4/5 | 15 | 90/100 | 90 % | — | -100,8 / -93,6 | 2,89 / 5,08 | 31,4 |
| 452 | F8CD (V2) | SF7 · 4/5 | 20 | 91/100 | 91 % | — | -99,4 / -92,4 | 4,33 / 5,55 | 31,4 |
| 452 | F8CD (V2) | SF10 · 4/5 | 10 | 49/50 | 98 % | 49 | -104,4 / -95,6 | -0,23 / 6,11 | 191,2 |
| 856 | F8CD (V2) | SF7 · 4/5 | 10 | 27/50 | 54 % | 27 | -109,9 / -98,6 | -3,86 / 3,4 | 31,4 |
| 856 | F8CD (V2) | SF7 · 4/5 | 15 | 30/50 | 60 % | 30 | -108,8 / -102,6 | -2,95 / 1,71 | 31,4 |
| 856 | F8CD (V2) | SF7 · 4/5 | 20 | 26/50 | 52 % | 27 | -109,7 / -105,1 | -4,55 / 0,52 | 31,4 |
| 856 | F8CD (V2) | SF7 · 4/8 | 20 | 46/50 | 92 % | 46 | -102 / -95,8 | 1,64 / 3,96 | 42,2 |
| 856 | F8CD (V2) | SF10 · 4/5 | 20 | 44/50 | 88 % | 44 | -108,5 / -103,6 | -4,5 / 2,1 | 191,3 |
| 1290 | F8CD (V2) | SF7 · 4/5 | 10 | 4/50 | 8 % | 4 | -113,3 / -101,3 | -7,44 / 2,25 | 31,4 |
| 1290 | F8CD (V2) | SF12 · 4/8 | 10 | 27/50 | 54 % | 27 | -118 / -111 | -14,8 / -2,33 | 873,5 |
| 1290 | F8CD (V2) | SF10 · 4/5 | 10 | 12/50 | 24 % | 12 | -116,8 / -108,1 | -12,35 / -0,63 | 191,3 |
| 1290 | F8CD (V2) | SF7 · 4/5 | 15 | — | — | 25 | -112,4 / — | -6,46 / — | — |
| 1290 | F8CD (V2) | SF10 · 4/5 | 15 | 43/50 | 86 % | 43 | -111,2 / -105,5 | -7,24 / 0,8 | 191,3 |
| 1290 | F8CD (V2) | SF7 · 4/5 | 20 | 39/50 | 78 % | 40 | -109,2 / -102,8 | -3,49 / 0,97 | 31,4 |
| 1290 | F8CD (V2) | SF10 · 4/5 | 20 | 47/50 | 94 % | 47 | -109,2 / -103,8 | -4,84 / 1,65 | 191,3 |
| 1696 | F8CD (V2) | SF7 · 4/5 | 15 | 0/31 | 0 % | — | — / — | — / — | — |
| 1696 | F8CD (V2) | SF7 · 4/5 | 20 | 0/22 | 0 % | — | — / — | — / — | — |
| 1696 | F8CD (V2) | SF10 · 4/5 | 20 | 5/50 | 10 % | 5 | -119,4 / -119 | -15,55 / -10,4 | 191,2 |
| 1696 | F8CD (V2) | SF10 · 4/8 | 20 | 5/50 | 10 % | 5 | -121,2 / -119,6 | -17,05 / -8,8 | 252,7 |
| 1696 | F8CD (V2) | SF12 · 4/8 | 20 | 6/25 | 24 % | 6 | -125,8 / -125,1 | -20,96 / -13,96 | 873,4 |
| 1696 | F604 (V4.3) | SF12 · 4/8 | 20 | 19/25 | 76 % | 21 | -116,2 / -108,5 | -12,29 / -7,8 | 874,3 |
| 2142 | F604 (V4.3) | SF12 · 4/8 | 25 | 9/25 | 36 % | 19 | -115,9 / -117,7 | -13,17 / -19,44 | 874,3 |
| 2142 | F604 (V4.3) | SF12 · 4/8 | 28 | 16/25 | 64 % | 18 | -115,4 / -116,2 | -11,86 / -15,64 | 874,3 |
| 2540 | F604 (V4.3) | SF12 · 4/8 | 28 | 0/10 | 0 % | 9 | — / — | — / — | — |

## A.2. Prueba de capacidad de octubre

**Tabla 10.** Prueba de capacidad: 300 paquetes de 239 bytes por punto, a 500 kHz y CR 4/5. La cota es la de Semtech para tramas sin separación; la separación es el tiempo entre el fin de una trama y el comienzo de la siguiente que agrega el firmware.

| Sentido | SF | Recibidos | PDR | Goodput (bit/s) | Cota (bit/s) | Relación | Separación (ms) | Aire medido − calc. (µs) | RSSI (dBm) | SNR (dB) |
|---|---|---|---|---|---|---|---|---|---|---|
| V2 a V4.3 | 7 | 300/300 | 100 % | 18068 | 19138 | 94,4 % | 5,95 | −30 | -30,8 | 9,95 |
| V2 a V4.3 | 8 | 300/300 | 100 % | 10468 | 10816 | 96,8 % | 5,92 | −31 | -30,5 | 11,01 |
| V2 a V4.3 | 9 | 300/300 | 100 % | 6003 | 6117 | 98,1 % | 5,96 | −32 | -30,1 | 10,3 |
| V2 a V4.3 | 10 | 300/300 | 100 % | 3297 | 3331 | 99 % | 5,95 | −35 | -29,4 | 8,4 |
| V2 a V4.3 | 11 | 300/300 | 100 % | 1819 | 1829 | 99,4 % | 5,96 | −40 | -30,1 | 5,9 |
| V2 a V4.3 | 12 | 299/300 | 99,7 % | 986 | 992 | 99,4 % | 5,94 | −50 | -29,6 | 3,56 |
| V4.3 a V2 | 7 | 300/300 | 100 % | 18419 | 19138 | 96,2 % | 3,65 | 246 | -28 | 6,87 |
| V4.3 a V2 | 8 | 300/300 | 100 % | 10587 | 10816 | 97,9 % | 3,59 | 246 | -32,1 | 7,71 |
| V4.3 a V2 | 9 | 271/300 | 90,3 % | 5458 | 6117 | 89,2 % | 3,6 | 246 | -34,8 | 6,54 |
| V4.3 a V2 | 10 | 300/300 | 100 % | 3309 | 3331 | 99,3 % | 3,61 | 245 | -34,8 | 6,61 |
| V4.3 a V2 | 11 | 300/300 | 100 % | 1822 | 1829 | 99,6 % | 3,63 | 245 | -35,3 | 5,99 |
| V4.3 a V2 | 12 | 294/300 | 98 % | 970 | 992 | 97,8 % | 3,63 | 243 | -26,5 | 6,1 |